BankSimFM: A MarS-Inspired Retail Banking Financial Distress Early-Warning Simulator

MH6818 FinTech Innovation with AI

Group Number: Group 9

Team Members:

SRIVASTAVA ABHISHEK (G2505156E)

WANG SHIQI (G2505241B)

YU CHENPU (G2506144E)

YAN LIYAO (G2506342D)

Live Demo: https://6818-group9-banksimfm.streamlit.app/

Submission Due Date: 29 May 2026

Executive Summary

Retail banks often identify customer financial distress only after a harmful event has already occurred, such as a missed payment, failed debit, or overdraft. This project proposes BankSimFM, a MarS-inspired retail banking sequence-modeling prototype that treats a customer's financial life as a time-ordered event stream rather than a static tabular snapshot. The system is designed to estimate 30-day distress risk, forecast likely future account behavior, and simulate intervention-conditioned outcomes.

BankSimFM adapts the logic of the MarS financial market simulation framework from market-order trajectories to customer financial-event trajectories. Instead of order flow, the model learns sequences of salary credits, bill payments, card spend, repayment events, overdraft activity, and support interactions. The prototype combines a causal transformer, an LSTM benchmark, a deterministic account-state engine, and a Streamlit dashboard for analyst-facing what-if analysis.

The latest trained system shows that the transformer is now the strongest primary live scorer, with test AUC 0.8342 and test F1 0.5828, compared with LSTM test AUC 0.8126 and test F1 0.5126. The project further supports intervention simulation, portfolio stress monitoring, fairness review, and collections prioritization. At the same time, the project remains a synthetic-data, course-project prototype rather than a production decision engine.

The main contribution of the project is demonstrating that a MarS-style financial foundation model can be adapted from markets to retail banking in a technically coherent and operationally meaningful way. The prototype supports early-warning use cases, scenario testing, and decision-support workflows while also surfacing governance considerations such as fairness, explainability, privacy, reliability, and human oversight.

1. Research Background and Problem Statement

1. The Practical Dilemma of Retail Banking Risk Management

Retail banking was selected as the focus domain because financial distress is both economically important and operationally observable through sequential account behavior. However, the current risk management in retail banks generally suffers from the problem of being reactive and lagging behind. Banks can see salary timing, recurring obligations, spending pressure, and credit utilization, but they rarely combine these signals to capture the path into distress. Traditional scorecards and static rules are useful for broad segmentation, but they may miss the temporal progression that precedes a missed payment or overdraft event. Banks usually react after negative events occur, by which time customers have incurred late fees and credit damage (Boonman et al., 2017). This underscores the need to shift from reactive response to proactive early warning.

2. Evolution and Research Gaps

From a historical perspective, financial distress detection methods have shifted from qualitative to quantitative. According to Beltman et al. (2025), early warning methods fall into three categories: expert judgment, analogous approaches based on historical cases, and parametric methods driven by data. The parametric approach has become mainstream due to its scalability, objectivity, and transparency, making it particularly suitable for retail banking's massive customer volumes.

Traditional credit scoring models relied on logistic regression and discriminant analysis, while recent machine learning methods such as random forests and gradient boosting have significantly improved predictive accuracy (Bequé & Lessmann, 2017; Barboza et al., 2017). For retail customer default prediction, a meta-model has been shown to outperform any single model, achieving an AUC above 0.80 (Beltman et al., 2025).

However, there is relatively few study on retail settings while the majority of studies mostly concentrate on business loans (Allen et al., 2004). This gap has multiple causes. First, existing studies predominantly use socio-demographic variables, such as marital status (Kočenda & Vojtek, 2009) or age and occupation (Leow & Crook, 2014), while neglecting richer behavioral and financial data. Also, existing EWS predominantly depend on historical data, leading to delayed intervention and risk mitigation (Beltman et al., 2025). This problem is particularly acute in retail settings, where financial conditions of customers change rapidly and require near-real-time monitoring (Barrell et al., 2010). Third, retail loans consist of numerous small, diverse transactions that can quickly alter a borrower's risk profile, requiring real-time data analysis to address emerging hazards (Beltman et al., 2025). Unlike corporations with

standardized financial statements, retail customers lack comparable quantitative information. Banks can only rely on high-frequency event data such as transaction records, payment behavior, and account balances.

3. Research Motivation

This project addresses late detection. A proactive system should identify at-risk customers before payment failure, overdraft, or persistent low balance occurs, and allow comparison of different interventions before acting. Customer financial behavior is naturally sequential: salary affects repayment ability, recurring bills reduce liquidity, balance deterioration raises risk, and interventions can shift outcomes.

BankSimFM addresses three core questions:

First, the lag problem: How to proactively identify customers likely to enter distress within 30 days? We employ high-frequency event data and a causal transformer for forward-looking early warning.

Second, the prediction problem: How to predict specific future behavioral trajectories instead of simple risk scores? We use autoregressive generation to output future events, balance paths, and utilization trajectories.

Third, the decision support problem: How to evaluate different interventions before acting? We use intervention-conditioned simulation to compare baseline versus multi-intervention strategies.

The BankSimFM is enables forward-looking early warning, trajectory prediction and intervention assessment through causal sequence modeling.

1.4 Innovation Thesis and MarS Alignment

The innovation thesis of BankSimFM is that a financial foundation model for retail banking can learn richer short-horizon risk dynamics than static summary models by operating directly on customer-event sequences. Static models reduce a customer to aggregate features such as average balance or utilization, losing the sequential order and causal relationships among events. In contrast, BankSimFM learns from the ordered interaction between incoming cash flow, recurring obligations, repayment stress, and account deterioration. As demonstrated in the MarS paper, sequence modeling captures complex dependencies in market order flow; the same logic applies to retail banking customer behavior.

The project is inspired by MarS: A Financial Market Simulation Engine Powered by Generative Foundation Model. MarS models financial markets as sequences of time-ordered events and uses a generative framework to support forecasting, detection, controllable simulation, and what-if analysis. MarS is built on three core concepts: high-resolution sequence modeling, controllable generation, and interactivity. BankSimFM adapts the same philosophy to retail banking. Rather than modeling order flow in a market, it models event flow in a customer's financial life. The MarS to BankSimFM mapping are shown in Table1.

MarS concept	BankSimFM equivalent
Order sequence	Customer financial-event sequence
Market trajectory	Customer cash-flow and distress trajectory
Generative sequence model	Causal transformer over banking events
Clearing mechanism	Deterministic account-state engine
Controllable generation	Intervention-conditioned simulation
Market-risk detection	Financial distress early warning
What-if market analysis	Intervention scenario analysis

Table 1. MarS to BankSimFM Mapping

BankSimFM is not a full reproduction of MarS. It is a simplified academic adaptation. The synthetic environment is much smaller, interventions are directional rather than causal, and the account-state engine remains the financial source of truth. Even so, the project preserves the most important conceptual elements: sequential modeling, controllable forecasting, path-dependent simulation, and analyst-facing interactivity.

2. Data Strategy and Preparation

This project adopts a hybrid data strategy, serving two purposes: specifying data requirements for real-world bank deployment and providing reproducible, privacy-free synthetic data for the course prototype. All prototype data is purely synthetic, containing no real personally identifiable information, and can be safely used for classroom demonstrations and open-source sharing.

1. Production Data Requirements

Deploying a similar system in a real banking environment would require the following four categories of data:

Category	Fields	Purpose
Customer profile data	Income band, employment type, region, product holdings, tenure, risk segment	Customer segmentation and repayment capacity assessment
Account and credit state data	Balance, overdraft limits, credit limits, utilization, delinquency status	Liquidity and credit pressure assessment
Event stream data	Salary credits, transfers, rent, utilities, card spend, loan due, loan payments, missed payments, ATM withdrawals, fees, support contacts	Core input for model training
Outcome labels	Overdraft, missed payment, persistent low-balance flags, delinquency stages, 30-day distress label	Supervisory signal

Table 2. Real Banking Environment Requiring Data

In production, banks would collect these data from core banking systems, card ledgers, repayment systems, collections systems, and customer-service logs, followed by preprocessing steps including de-identification and access control, timestamp normalization, missing-value handling, event taxonomy standardization, and customer-level train/validation/test splitting.

2. Prototype Data Strategy

Since real banking data is inaccessible, this project implements a synthetic data generator. This generator is a probabilistic model based on behavioral rules, not simple random simulation. It creates five customer archetypes, each with distinct behavioral characteristics: stable salaried (fixed paydays, regular expenses, savings buffer), volatile income (unstable pay timing and amounts), high obligation (high rent/loan burden), rising utilization (continuously increasing credit card usage), and near distress (already experiencing overdrafts or partial delinquencies).

These archetypes are not fixed templates. The generator samples customer-specific traits including income level, volatility, rent burden, spending intensity, repayment tendency, recovery tendency, starting buffer, and credit usage, ensuring that customers within the same archetype still exhibit individual differences. Under the current packaged artifact configuration, the generator produces 300 synthetic customers and 82,754 event rows.

The synthetic customer metadata also includes fairness-ready segment fields: income band, employment type, region, and risk segment.

3. Event Schema and Labeling

Each event contains both event and state information. Key fields include:

Field	Description
customer_id	Synthetic customer identifier

event_timestamp	Time of event
event_type	Event class
amount	Event amount
amount_direction	Credit or debit
category	Merchant or event category
balance_before	Account balance before event
balance_after	Account balance after event
credit_limit	Credit limit
credit_utilization	Utilization ratio
days_to_next_due	Days to next scheduled repayment
intervention_flag	Intervention marker
distress_label_30d	Distress within next 30 days

Table 3. Key fields and Description

The distress label adopts a forward-looking implementation definition. In the current prototype, a customer is labeled as entering financial distress within the next 30 days if the future event stream contains any of the following implemented distress events: `loan_emi_missed`, `failed_debit`, or `overdraft_event`. This definition is narrower than a broader business concept of distress, but it is the exact labeling rule used by the current synthetic-data pipeline and model training setup.

4. Data Preprocessing and Training Sample Construction

The preprocessing pipeline sorts each customer's events strictly by time, derives sequence features, buckets selected continuous values, and constructs fixed-length history windows. The split is done at the customer level rather than the row level, which reduces leakage across train, validation, and test sets.

The current pipeline also carries internal due-state information so that restructuring-style interventions are visible to the model. Intervention-augmented windows are generated for the transformer to strengthen learned conditioning on simulated support actions.

An illustrative training sample structure is shown below.

Component	Example
Context window	Last 256 customer events
Event tokens	salary_credit -> rent_payment -> card_spend -> credit_card_payment_due -> card_spend
Dense features	balance, utilization, days to next due, due-amount feature, intervention flag
Primary targets	next event, next amount bucket, next balance-delta bucket, distress label
Intervention token	none, reminder, due_date_shift_7d, temporary_overdraft_buffer, or installment_restructure

Table 4. Training Sample Structure

This data strategy is based on the following assumptions. First, the distribution of synthetic data can reasonably approximate the main patterns of real retail banking customer behavior. Second, training, validation, and test sets are split by customer with no event-level data leakage. Third, last 256 historical events are sufficient to capture customer risk signals. Fourth, missing values can be handled through deterministic defaults.

3. Model Architecture and Training Approach

3.1 Architecture Rationale: From MarS to BankSimFM

The architecture of the BankSimFM model is based on the MarS paper, which introduces a financial market simulation engine built upon a generative foundation model. The Large Market Model: MarS learns from fine-grained order-level market data and generates future market trajectories under different conditions. The paper discusses three major requirements to a useful financial simulator, namely realism, controllability and interactivity. MarS utilises the historical order sequences, user-injected orders, and market conditions as the input of conditional generation.

BankSimFM takes this idea from financial markets into retail banking. Unlike the model of order level trading behaviour, BankSimFM models customer level banking events. Each customer history is treated as a sequence of financial events such as salary credits, debit transactions, repayment behaviour, failed payments, overdraft events and changes in account balance or credit utilisation. The objective is to build a prototype in the foundation-model style that can learn customer behaviour patterns, predict future events, and quantify the risk of financial distress.

The design also directly addresses the requirement to propose an appropriate architecture for the foundation model, explain how the model would be trained, describe how training data is constructed and fed into the model, and define how performance would be evaluated. The project also provides for actual prototype training as mentioned in the assignment brief for the additional-credit opportunity.

3.2 Main Model: Causal Event Transformer

The main model in BankSimFM is a Causal Event Transformer. It is designed to learn from ordered customer-event histories and generate predictions based on the most recent customer state. This is similar to the MarS logic of using sequence modeling to generate future financial trajectories, but the unit of modeling changes from market orders to banking customer events.

Each customer event is represented through several embedded components:

- event type;
- transaction amount bucket;
- account balance bucket;
- credit utilization bucket;
- due amount bucket;
- time gap bucket;
- intervention type;
- positional information.

These embeddings are summed to form the model input at every time step. This sequence is then processed by a Transformer encoder with multi-head attention and feed-forward layers. The model applies a sequence mask such that padded positions are masked. The model encodes the sequence and then uses the last valid time step for customer-level representation for prediction.

This architecture is suitable for retail banking as customer distress is usually not caused by a single isolated event. It is often the result of a pattern of events over time. For example, a customer may see increasing credit utilisation, lower balances, repeated failed debits and then a missed repayment. A Transformer can learn these event interactions in a more flexible manner than a static classification model as it can attend to different parts of the customer history.

3.3 Multi-Task Prediction Design

BankSimFM does not train the Transformer only as a binary classifier. The model has four prediction heads:

1. a next-event prediction head;
2. a next-amount-bucket prediction head;
3. a next-balance-delta-bucket prediction head;
4. a distress-risk prediction head.

This multi-task design is important because the model is expected to both forecast and score. The next-event head helps the model learn what kind of banking event can follow. The amount and balance-delta heads enable the model to learn the financial size and account-state impact of future events. The distress head maps the learned representation of the sequence to a risk score.

This structure is in line with the foundation model logic in MarS. In MarS, the model produces not only a single direct forecast target but also the simulated trajectories for forecasting, detection, what-if analysis, and agent training. BankSimFM learns customer-event dynamics, enabling the same trained model to power downstream tasks such as early distress warning, intervention testing, collections prioritisation, and portfolio stress monitoring.

3.4 Baseline Model: LSTM Distress Classifier

The project also includes an LSTM distress model for benchmarking. The LSTM takes as input the numerical customer-event features in dense form and sequentially processes the features, with the last valid hidden state used to predict the distress with a linear classifier.

The LSTM is used as it is a standard benchmark for sequence based financial prediction. It can represent time-ordered customer behaviour, but it compresses the sequence in steps. Since self-attention enables modelling relationships between different points in customer history, we expect the Transformer to be more flexible. Including the LSTM baseline also improves the evaluation design, as it enables the project to show if the proposed foundation-model-style architecture adds value over a simpler sequential model.

3.5 Training Data Construction

The training data is in the form of customer-level event sequences. Each sample consists of a fixed length historical window and one or more prediction targets. The input history is tokenised and bucketed to features for the Transformer. The same customer histories are also transformed into dense numerical features for the LSTM benchmark. This allows the two models to be trained and evaluated on the same train, validation and test splits.

For the Transformer, each training batch contains:

- sequence tokens;
- sequence mask;
- next event target;
- next amount bucket target;
- next balance-delta bucket target;
- distress target;
- sample weights.

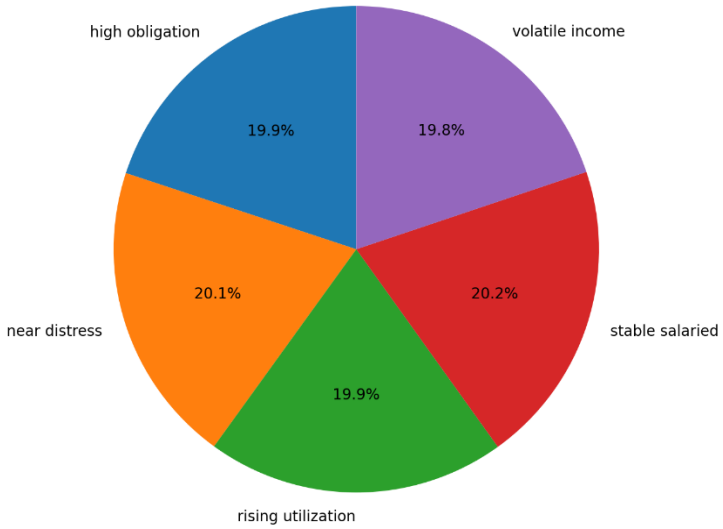

Figure 1. Test Sample Composition by Customer Archetype

The LSTM input consists of dense features, a mask, the distress target, and sample weights for each batch. This is required because different customers have different length histories and the model should not learn from the padded values.

The checklist also requires the report to explain how the model makes use of intervention-aware features. In this implementation, the intervention type is embedded in the token representation of the Transformer. This means the model can be conditioned on scenarios like no intervention, payment support, or other customer-treatment actions. This design supports the controllability requirement adapted from MarS, where the user-injected actions are part of the conditional generation process.

3.6 Training Objective

The Transformer is trained with a multi-task objective. The total loss combines next-event prediction, amount-bucket prediction, balance-delta prediction, and distress classification:

$$L_{total} = L_{event} + 0.5L_{amount} + 0.5L_{delta} + \lambda L_{distress}$$

where (L_event}), (L_amount}), and (L_{balance delta}) are cross-entropy losses, and (L_{distress}) is a weighted binary cross-entropy loss. The weighting is important because financial distress events are less frequent than normal customer events, so the model needs to reduce the effect of class imbalance during training.

The LSTM baseline uses a simpler training objective. It is trained only on the binary distress target using weighted binary cross-entropy. This makes the LSTM a clean benchmark for distress-risk prediction, while the Transformer is trained as a broader event-forecasting and risk-scoring model.

3.7 Training Process

Both models are trained with mini-batch training using PyTorch DataLoaders. The training pipeline first creates the datasets and the model objects, then sends them to the available device, and trains them with the Adam optimiser. The code will automatically select the available device, so the prototype can run on CPU for small demo data, and GPU acceleration is preferred for larger training runs. The Transformer and LSTM are trained on the same training split in each epoch. The validation AUC is calculated after each epoch. The best model checkpoint is saved when the validation auc improves We use a patience mechanism for early stopping, so training can be stopped when both models are no longer improving. This keeps overfitting low and avoids computation that is not needed.

The saved model artifacts include:

- transformer.pt;
- lstm.pt;
- metrics.json;
- fairness_metrics.json;
- simulation_metrics.json.

This artifact-based design supports reproducibility because the trained checkpoints and evaluation outputs can be reused by the dashboard and final report.

3.8 Evaluation Metrics

The main predictive evaluation focuses on binary distress classification. After training, the best checkpoints are reloaded. The validation set is used to select the best classification threshold, and the test set is used for final evaluation. The reported metrics include:

- AUC;
- precision;
- recall;
- F1 score;
- accuracy;
- selected threshold.

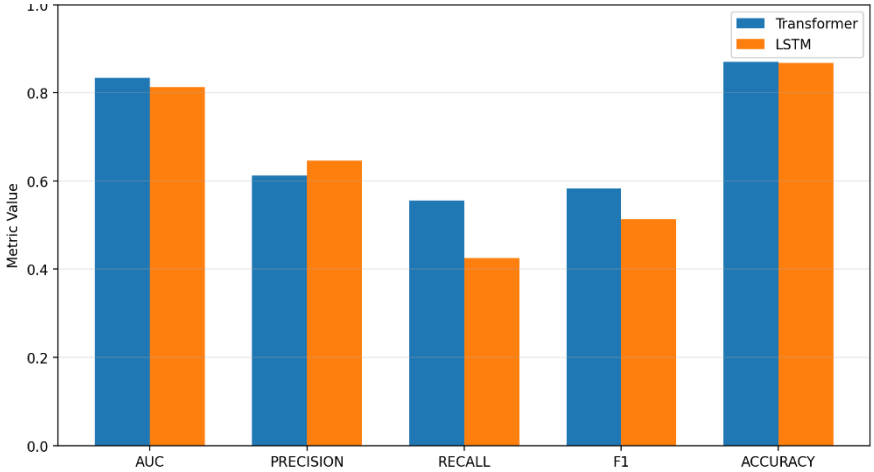

Figure 2. Test Metrics: Transformer vs LSTM

The AUC score indicates the model's ability to rank risky vs. non-risky customers. Precision tells us whether the customers we label as risky are indeed risky. Recall is a measure of how well the model identifies distressed customers. F1 is a trade-off between precision and recall. Accuracy gives a global evaluation of the classification, but is not informative in the case of unbalanced class distribution.

The evaluation design also comprises additional metrics mandated by the checklist. The training pipeline produces fairness metrics across customer segments such as archetype, income band, employment type, region and risk

segment. It also generates early warning and simulation metrics such as average time to distress, false-positive rate for stable customers, event-mix realism, balance trajectory realism, stability across repeated simulations and intervention usefulness.

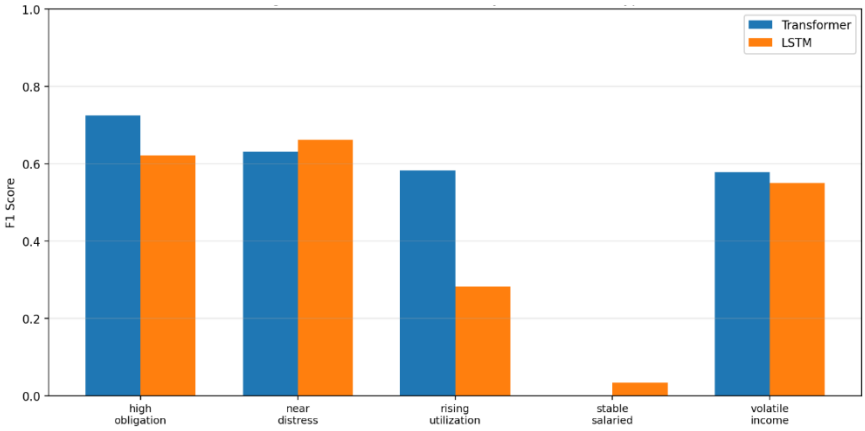

Figure 3. Fairness Breakdown by Customer Archetype

These metrics are important because the project requires not only technical model evaluation, but also strategic assessment of model reliability, fairness, governance, and business usefulness.

3.9 Scalability and Infrastructure Assumptions

The prototype is for local training on synthetic data but the architecture can scale to larger production banking data sets. In a real bank, the same model design could be trained on larger customer-event histories, more product types and longer time horizons. This is in line with the scaling laws discussed in MarS, which indicates that larger datasets and larger models can improve the performance of financial foundation models.

CPU training is acceptable for this project as demonstration. GPU training is preferred for larger experiments. The cost of training Transformers increases with the context length, hidden size, number of layers and number of customers. If the shape of the model is changed (e.g. hidden size, number of heads, bucket definitions, output heads), the model needs to be retrained as older checkpoints might not match the new architecture.

3.10 Summary

In summary, the BankSimFM model architecture implements the MarS foundation-model idea in a retail banking scenario. MarS is a generative sequence model to model order-level financial market time series. BankSimFM uses a Causal Event Transformer to learn trajectories of customers and events, predict future account activities and assess risk of distress. LSTM baseline gives a clear benchmark, and multi-task structure of Transformer supports richer downstream applications. This makes the architecture a good fit for the project, linking technical foundation model design with practical financial-service innovation.

4. Forecasting, Scoring, and Simulation Interfaces

4.1 Interface Design Rationale

BankSimFM provides three public interfaces for inference: score_customer, forecast_customer and simulate_intervention. These interfaces operationalize the trained customer-event model into concrete retail banking functions. The design adheres to the MarS logic, where a generative financial foundation model not only performs one direct prediction task, but also facilitates forecasting, detection, what-if analysis, and interactive simulation. In MarS, users can assess potential future market conditions through simulated trajectories. BankSimFM brings such an idea

into retail banking by simulating paths of events on the customer level and comparing customer risk under different intervention scenarios.

This section also provides the project's requirement that the project should describe how a trained financial foundation model can support downstream applications and translate model outputs into financial-service decisions.

4.2 Public Inference API

The BankSimFM package exposes three main functions through its public API: forecast_customer, score_customer, and simulate_intervention. These functions are imported in the package-level __init__.py, which makes them available as the core inference layer of the prototype.

The three functions have different roles:

- score_customer(history, horizon_days=30) estimates the customer's financial distress probability.
- forecast_customer(history, horizon_days=30) generates a future customer-event path.
- simulate_intervention(history, intervention_type, horizon_days=30) compares a baseline scenario with an intervention-adjusted scenario.

Together, these interfaces form the operating layer of BankSimFM. The model output is not limited to a single probability score. Instead, the system produces risk scores, event forecasts, balance paths, utilization paths, negative-event summaries, and scenario comparisons. This makes the prototype suitable for dashboard-based analysis and business decision support.

4.3 Customer Scoring Interface

The score_customer() function takes a customer's historical banking events as input and returns a structured ScoreResult. The output includes the predicted distress probability, a binary distress label, top risk drivers, and recent risk signals.

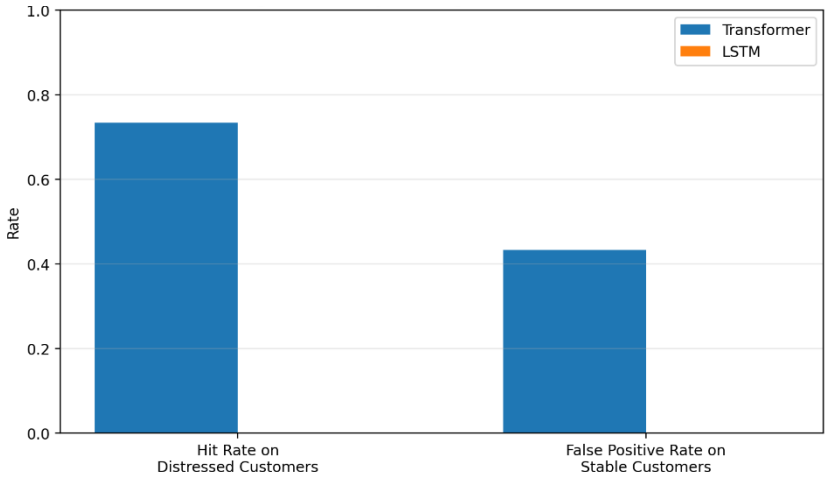

Figure 4. Early-Warning Classification Rates

The function normalizes the input history prior to scoring. Can accept a pandas DataFrame, or a list of event dicts. The history is transformed to a DataFrame, timestamps are parsed and events sorted by time. This means that the model always gets a customer history in order.

If a trained Transformer checkpoint is available, the scoring function uses the Transformer to estimate probability of distress . The model is conditioned on the customer's event history, and the default intervention is "no intervention". If

there is no trained model, then the function resorts to a heuristic score based on rules. This fallback adds some robustness to the demo, but the preferred path is the model-driven scoring with the trained Transformer.

Then the predicted probability is compared against the selected Transformer threshold to predict the distress label. If trained artifacts exist, this threshold is pulled from the saved model metrics. The function also affects the probability for longer forecast horizon, e.g. 60 or 90 days, which is consistent with the idea that risk exposures might increase over a longer horizon.

4.4 Forecasting Interface

The forecast_customer. function generates a forward-looking customer-event path for a specified horizon. In the prototype configuration, the default horizon is 30 days, with allowed dashboard horizons of 30, 60, and 90 days.

When trained model artifacts are available, forecasting is model-driven. The function calls decode_forecast using the trained Transformer, the customer history, the selected horizon, and the model context length. The current implementation uses a greedy decoding strategy for the public forecast interface.

The forecast output is returned as a ForecastResult, which contains:

- projected future events;
- projected account balance path;
- projected credit utilization path;
- a forecast summary.

The reason this design is useful is that retail banking risk is not just about whether a customer is risky. It is also important to understand how the customer can become a risk. For instance, the forecast might suggest that the customer's balance is expected to decrease, credit utilization is expected to increase, or that negative events such as failed debits or missed repayments are likely to occur. These future paths form the basis of early-warning analysis and customer monitoring.

If no trained artifacts are available, the function resorts to a heuristic fallback prediction. In this fallback mode the system creates simple future events using deterministic account state logic. But this is only a back-up route. The final prototype aims to make the main forecasting path be driven by the Transformer sequence model instead of just static rules.

4.5 Intervention Simulation Interface

The function simulate_intervention allows for what-if analysis. It compares a forecast baseline to a forecast adjusted for an intervention. This is one of the most important interfaces as it links the model to real world banking decisions.

First, the function checks that the chosen intervention type is supported. It then builds a baseline forecast using the original customer history. It then translates the customer history into an account state, applies the selected intervention policy and writes the altered state into the customer history. This modified history is then fed into the Transformer based forecasting process.

This makes the design more robust than changing an intervention token. The intervention-adjusted state is directly fed to forecasting and simulation. Hence the model receives both the intervention condition and the modified account state. This more accurately captures the business impact of an intervention. For example, the change of a label in a payment support intervention should be accompanied by a change in the customer’s due-state, balance condition or repayment pressure before subsequent events are simulated.The output is returned as a SimulationResult. It contains

both the baseline and intervention sides, each with its own risk score and forecast result. It also includes the risk delta and a list of scenario differences. These differences summarize how the intervention changes the distress probability, ending balance, ending utilization, and projected negative-event count.

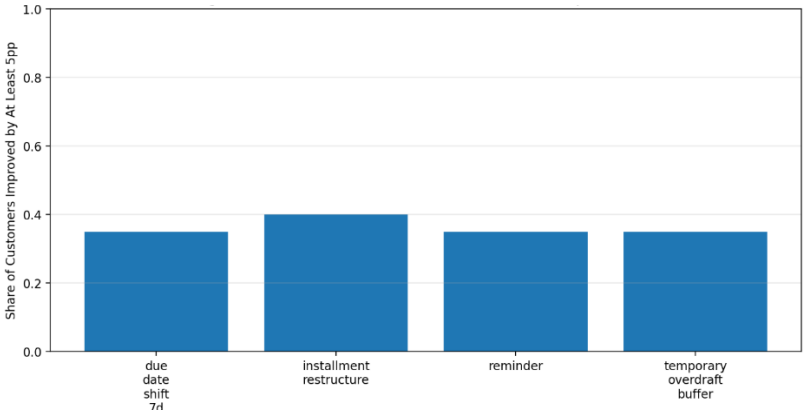

Figure 5. Share of Customers with Material Risk Improvement

This interface directly adapts the MarS what-if logic to retail banking. In MarS, users can inject actions into the market simulator and observe how simulated market trajectories change. In BankSimFM, the user selects an intervention and observes how the customer's projected event path and distress risk change.

4.6 Forecast Summary and Negative-Event Analysis

The forecasting interface also returns a forecast_summary. This summarization is important because decision makers should not have to manually inspect each generated event. Instead, the system extracts the most salient risk signals along the projected path.

The forecast summary includes the forecasted number of negative events, the top negative events, path-level risk information and scenario-level metrics like the ending balance, ending utilization and negative-event share.

This design improves requirement coverage, since the output does not only show raw simulation results. It turns projected events into an understandable risk summary. For example, a bank user can quickly see if a customer's forecast shows repeated failed debits, missed payments, overdraft events, or rising utilization. This enhances the usability of the simulation result for customer monitoring, collections prioritization and early intervention decisions.

4.7 Model Loading and Fallback Logic

The inference layer is intended to use saved model artifacts, when available. The function _load_trained_models loads the trained Transformer, LSTM benchmark, chosen thresholds and device configuration from saved artifacts. If the necessary artifacts are missing or incompatible, the system can trigger training via ensure_demo_artifacts.

This design allows reproducibility. The dashboard and inference functions don't need to retrain the model every time. They can reuse saved checkpoints like transformer.pt, lstm.pt, metrics.json, fairness_metrics.json, and simulation_metrics.json.

The fallback logic also improves robustness. Without trained models, heuristic approaches can still be used for scoring and forecasting. However, the model-driven path is the primary design for the final prototype as it better fulfills the project goal of creating a financial foundation-model-style system.

4.8 Runtime and Infrastructure Assumptions

The runtime layer automatically selects the best available computation device. It prefers Apple Metal Performance Shaders on Apple Silicon, then CUDA GPU, and finally CPU.

This supports flexible deployment for a course prototype. Small synthetic-data demos can run on CPU. Apple Silicon can be used when MPS acceleration is available. CUDA GPU is preferred for larger training or repeated simulation workloads. These assumptions are consistent with the model configuration, which uses a context length of 256, hidden size of 256, four Transformer layers, eight attention heads, and a default forecast horizon of 30 days.

4.9 Business Usefulness of the Interfaces

The three interfaces support different business decisions in retail banking:

Firstly, score_customer allows early warning of distress. The probability of distress and top drivers can help a bank identify customers who may need attention before they miss payments or fail to pay their accounts.

Second, forecast_customer helps you keep track of customers easily. Instead of a static score, analysts can examine the projected path of the balance, utilization and negative-event pattern. This helps to explain why a customer may turn risky.

Third, simulate_intervention allows you to test policies. Before acting, a bank can compare a baseline scenario with a treatment scenario. For example, it can predict if a payment support intervention could reduce risk, increase an ending balance, decrease utilization or reduce projected negative events.

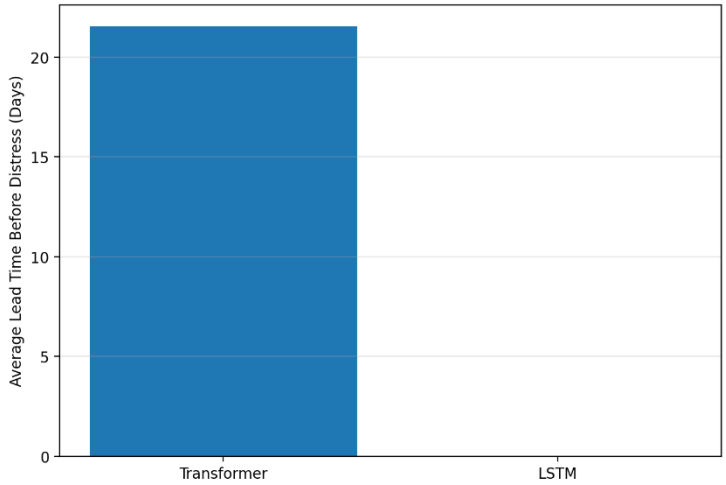

Figure 6. Average Early-Warning Lead Time

These outputs can support operational decisions such as the prioritization of customer outreach, the ranking of collections queues, the assessment of portfolio stress, and the design of interventions. They also connect the technical model outputs and economic value measurement, since the model can estimate how risk and negative events change under different scenarios.

4.10 Limitations and Governance Notes

The simulation results are to be viewed as directional model outputs and not causal proof. The intervention simulation demonstrates how projected outcomes change under model conditioned scenarios, but it doesn’t prove the intervention will cause the same effect in production. This is important for responsible use.

The prototype also uses synthetic data for illustration. Therefore, the outputs should not be used for real-time credit decisions, automated adverse action or customer level financial treatment without further validation. In a real-world

setting, the model would need to be validated on production data, fairness tested, monitored, human reviewed and have governance controls.

This limitation conforms to the general caution typically seen in financial simulation systems. Simulations are also introduced by MarS as a tool for analysis and controlled testing, not as a substitute for professional financial judgment.

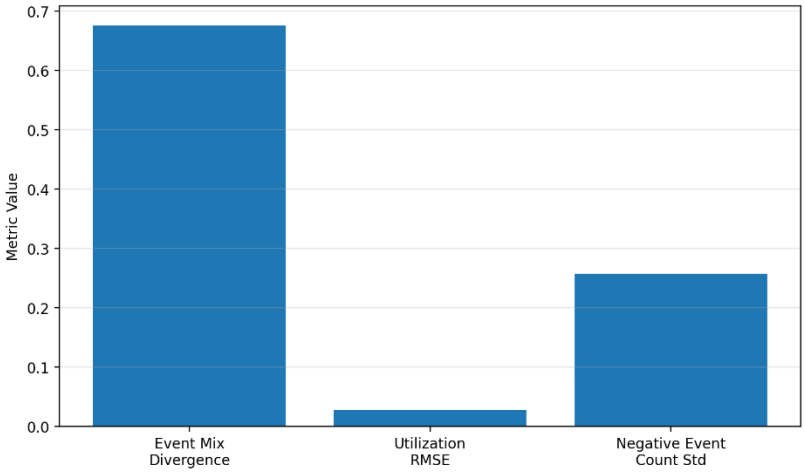

Figure 7. Simulation Quality and Stability Metrics

4.11 Summary

In short, the BankSimFM inference layer converts the trained sequence model into practical retail banking tools. score_customer provides distress-risk scoring, forecast_customer generates future customer-event trajectories, and simulate_intervention allows intervention-based what-if analysis. The design satisfies the checklist criteria, with scoring, forecasting and simulation implemented as public APIs. Forecasting is model-driven, when trained artifacts are present; intervention simulation is conditioned on both model tokens and adjusted account state; forecast summaries offer clearer information about projected negative events. Therefore, the prototype is suitable for technical demonstration and business-oriented decision support.

5. Evaluation and Results

The project evaluates both models on the same 30-day distress task and also generates supporting artifacts for early warning, fairness, simulation realism, intervention usefulness, and repeated-run stability. The following sections present each evaluation result in detail.

5.1.Holdout Classification Results

The performance comparison between the Transformer model and the LSTM baseline on the holdout test set is shown in the table below:

Model	Test AUC	Test Precision	Test Recall	Test F1	Test Accuracy
Transformer	0.8342	0.6127	0.5556	0.5828	0.8696
LSTM	0.8126	0.6458	0.4250	0.5126	0.8675

Table 5. Transformer model and the LSTM baseline

As shown in the results, the Transformer outperforms the LSTM in both AUC and F1 score, indicating superior ranking ability in distinguishing distressed customers from non-distressed ones, as well as better overall balance between precision and recall. Notably, while the LSTM achieves slightly higher precision while its recall score 0.4250 is substantially lower than 0.5556 from Transformer. This means that although a higher proportion of customers flagged

as risky by the LSTM are actually distressed, the LSTM misses significantly more true distressed customers. Considering the most important composite indicators AUC and F1, the Transformer has been selected as the primary live scorer for this prototype.

5.2.Early-Warning and Scenario Metrics

The current simulation artifact reports the following Transformer early-warning summary.

The average lead time before distress events is 21.55 days, meaning the model can issue warning signals approximately three weeks before a customer actually enters distress, which providing banks with a valuable window for intervention. The hit rate on distressed customers is 0.7333, indicating that among all customers who actually entered distress, and the model successfully identified approximately 73%. The false positive rate on stable customers is 0.4333, meaning approximately 43% of stable customers were incorrectly flagged as risky. This relatively high false positive rate reflects the inherent trade-off in retail banking early warning tasks between not missing high-risk customers and not falsely flagging low-risk customers.

Regarding simulation quality and stability, the system provides several metrics. The event-mix divergence is 0.675, which measures the difference in event type distribution between generated and real events, with lower values indicating greater similarity. The balance-trajectory RMSE is 371.85. The utilization RMSE is 0.0282, a relatively small value indicating high fidelity in simulating credit utilization behavior. The ending-balance standard deviation across repeat runs is 75.43, reflecting simulation stability. The negative-event-count standard deviation is 0.257, also used to measure simulation stability. These values show that the prototype is capable of generating non-trivial scenario variation while staying anchored to a deterministic financial-state engine.

5.3.Intervention Usefulness

The system evaluated the average 30-day risk reduction for different interventions. The results are presented in the table below:

Intervention	Avg Predicted Risk Reduction	Share Improved By 5pp
due_date_shift_7d	0.0171	0.35
installment_restructure	0.0228	0.40
temporary_overdraft_buffer	0.0217	0.35
reminder	-0.0447	0.35

Table 6. Effectiveness of Intervention

The data above show that installment restructuring achieves the largest average risk reduction at 0.0228, with 40% of customers experiencing risk improvement of 5 percentage points or more. Temporary overdraft buffer achieves an average reduction of 0.0217, with 35% of customers showing significant improvement. Due date shift of 7 days achieves an average reduction of 0.0171, also with 35% of customers showing meaningful improvement.

Notably, the reminder intervention shows a negative average risk reduction of -0.0447, meaning that risk increased rather than decreased on average. This result should not be interpreted as a code bug. Instead, it demonstrates that not all interventions are equally effective in the current synthetic environment. Lightweight reminder-based outreach may provide limited help to the most stressed customer segments, and may even have negative effects if delivered with poor timing or inappropriate messaging. In contrast, stronger structural interventions such as installment restructuring or temporary liquidity support can more substantially change a customer's financial situation, producing more significant risk reduction. This finding also provides data-driven support for banks' intervention strategy selection that different customer segments may require differentiated intervention approaches.

5.4 Fairness and Segment Behavior

The project also computes subgroup metrics across multiple dimensions to evaluate model performance consistency across different customer groups, including customer archetype, income band, employment type, region, and risk segment.

Subgroup performance is informative but uneven. For example, the transformer performs particularly strongly on several higher-risk groups, achieving an AUC of 0.9175 for the high obligation archetype and 0.9105 for the rising

utilization archetype. This indicates that the model can effectively identify distressed customers within these high-risk segments. At the same time, performance varies across regions and employment types, which means subgroup monitoring would remain necessary in any serious deployment path.

The report should interpret these fairness results cautiously. They are useful for governance discussion, but they are still shaped by a synthetic data generator rather than real customer populations. Before deploying in a real banking environment, fairness assessment must be re-conducted on real customer data to ensure the model does not introduce systematic bias against any particular group.

6. Downstream Applications and Economic Decision Translation

The trained BankSimFM prototype supports a coherent downstream workflow rather than a single isolated use case. The public interfaces score_customer, forecast_customer, and simulate_intervention allow the same learned model family to move from observation to prediction to operational comparison. This is where the foundation-model framing becomes economically meaningful.

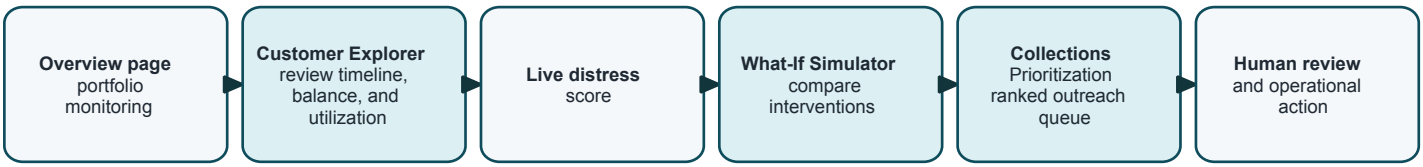

Figure 8. Analyst-facing workflow across scoring, simulation, and collections prioritization.

6.1 Early Distress Warning

The first downstream application is **early distress warning**. The transformer can score an observed customer history and identify customers who are likely to face distress within the next 30 days. In operational terms, this supports earlier review, earlier outreach, and reduced reliance on purely reactive collections activity after a failure event has already occurred.

6.2 Intervention What-If Analysis

The second application is **intervention what-if analysis**. The simulator can compare the baseline projected path with several intervention-conditioned futures, including due_date_shift_7d, temporary_overdraft_buffer, installment_restructure, and reminder. In the current artifact bundle, installment_restructure produces the strongest average predicted risk reduction at 0.0228, followed by temporary_overdraft_buffer at 0.0217 and due_date_shift_7d at 0.0171. Reminder is negative on average, which is strategically useful because it suggests that low-cost outreach is not always the best action for stressed customers.

6.3 Portfolio Stress Monitoring

The third application is **portfolio stress monitoring**. Because the synthetic customer table includes archetype and segment metadata, the dashboard can aggregate model outputs across groups such as income band, employment type, region, and risk segment. This provides a way to detect where distress pressure is building and where support capacity or policy design may need to shift.

6.4 Collections Prioritization

The fourth application is **collections prioritization**. The dashboard ranks customers by current transformer-based live risk, estimates the projected post-intervention risk, and recommends the best available intervention under the current scenario logic. This converts model output into a daily analyst workflow rather than a passive reporting metric.

6.5 Translation into Operational and Economic Decisions

Application	Operational decision	Business value mechanism
Early warning	Review or contact customers earlier	Avoid missed-payment losses and downstream delinquency costs
What-if simulation	Select best support action	Improve intervention targeting and reduce wasted outreach
Portfolio monitoring	Watch vulnerable cohorts	Improve segment-level planning and staffing
Collections prioritization	Rank customers for action	Increase analyst productivity and intervention efficiency

Table 7. Translation from downstream application to operational and economic decisions

The key translation into economic decisions is therefore straightforward. Early-warning scores influence which customers should be reviewed first. Scenario analysis influences which intervention should be chosen. Portfolio monitoring influences staffing and policy focus. Ranked prioritization influences how analysts allocate limited outreach capacity. The model is valuable not because it predicts in the abstract, but because it changes the timing, targeting, and expected yield of operational decisions.

7. Risk, Challenges, and Governance

The main governance benefit of the prototype is its synthetic-only data design. The packaged artifacts do not require real PII and therefore avoid many classroom-sharing risks that would arise with actual customer records. In a production setting, however, the bank would still need strong controls over data minimization, access rights, lineage, encryption, retention, and development environments.

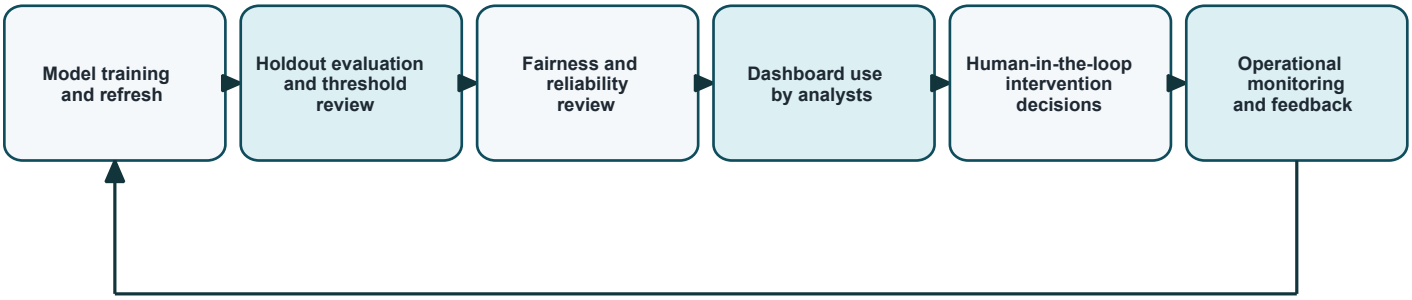

Figure 9. Governance and monitoring control loop for a responsible deployment path

The project surfaces several important governance themes.

7.1 Privacy

The current prototype uses synthetic data only, which reduces privacy risk materially. No real PII is stored in the repository or required for the classroom demo. In a production bank setting, strict controls would still be needed around data access, lineage, encryption, retention, and model-development environments.

7.2 Fairness

Fairness is addressed by including segment metadata and computing subgroup metrics across several segment dimensions. The results show that subgroup behavior is not uniform. This is a useful governance signal, but it should not be overinterpreted because the subgroup structure is synthetic and therefore does not guarantee real-world representativeness.

7.3 Explainability

The system supports partial explainability through:

- Recent risk-signal summaries
- Top-driver heuristics
- Timeline review in the dashboard
- Explicit baseline versus intervention comparison

This is not full causal explanation, but it gives analysts more context than a raw distress score alone.

7.4 Reliability and Operational Risk

The deterministic account-state engine improves simulation consistency by grounding decoded futures in explicit balance and utilization updates. Even so, the system remains a prototype and should not be treated as a production-grade decision engine. Operational risks include distribution shift, calibration drift, subgroup instability, and misuse of intervention outputs as if they were causal evidence.

7.5 Controls

If a real bank were to pursue this idea, the following controls would be necessary:

- Human-in-the-loop review for high-impact actions
- Ongoing fairness monitoring
- Periodic retraining and threshold review
- Monitoring for drift and intervention instability
- Strict separation between advisory analytics and autonomous adverse-action decisions This prototype should not be used for live autonomous credit decisions.

8. Measurement of Economic Value

The project does not claim realized bank value. Instead, it proposes a measurement methodology that connects model outputs to business decisions. The value story comes from earlier identification of distressed customers, better targeting of support actions, more efficient analyst triage, and lower wasted outreach on low-yield cases.

8.1 Value Drivers

There are four practical value drivers. The first is avoided loss from missed payments, failed debits, overdrafts, and downstream delinquency. The second is improved intervention efficiency, meaning a higher share of outreach cases produce measurable risk reduction. The third is analyst productivity, because ranked triage reduces time spent reviewing low-priority customers. The fourth is customer retention and relationship value, especially if timely restructuring or liquidity support prevents avoidable customer deterioration.

8.2 Scenario-Based Measurement Framework

The recommended measurement framework is scenario-based. A bank would estimate the number of eligible customers, the fraction that receive action, the incremental intervention success rate relative to current practice, the average avoided loss per prevented distress event, the analyst time saved per case, and the cost of each intervention. Value can then be estimated as avoided losses plus operating savings plus retention benefit minus action cost.

Use case	Decision lever	Value driver	Example measurement
Early warning	Trigger earlier review	Lower missed-payment loss	Reduction in missed-payment incidence among contacted cases
Intervention selection	Choose more effective intervention	Better intervention ROI	Average risk reduction per intervention type

Portfolio monitoring	Shift staffing or policy focus	Lower operational surprises	Distress trend by segment and action rate
Collections prioritization	Rank customers for outreach	Higher analyst efficiency	Cases reviewed per analyst hour and avoided downstream loss

Table 8. Proposed framework for measuring economic value

In a real implementation, the bank would estimate value by combining historical base rates, intervention costs, analyst capacity, and outcome deltas. The current project provides the decision-support structure, not a realized business case.

9. Assumptions, Limitations, and Future Work

9.1 Assumptions

This report rests on several important assumptions:

- The prototype uses synthetic data only
- The default distress horizon is 30 days
- The environment is prototype-scale rather than production-scale
- Intervention outputs are directional and scenario-based, not causal proof

9.2 Limitations

The project also has meaningful limitations:

- The synthetic world remains simpler than real retail banking behavior
- Fairness results depend on the synthetic segment design
- The intervention simulator is informative but not validated on real customer outcomes
- The work does not yet include a multi-seed stability study
- The dashboard and artifacts are designed for coursework demonstration rather than enterprise deployment

Despite these limitations, BankSimFM satisfies the core requirements of the assignment well. It explains why foundation models can drive innovation in retail banking, defines a realistic data strategy, implements a MarS-inspired sequence architecture, compares transformer and LSTM training approaches, demonstrates downstream business applications, addresses governance concerns, and proposes a defensible methodology for economic value measurement. The strongest conclusion is that the project moves beyond a concept note: it shows a working prototype that connects foundation-model design to concrete financial-services decisions.

9.3 Future Work

Recommended future work includes:

- Training and validation on real or more richly calibrated data
- Formal calibration analysis
- Repeated-seed stability testing
- Additional intervention types
- Longer-horizon forecasting studies
- Deeper economic-value modeling with explicit cost assumptions

10. Conclusion

BankSimFM demonstrates that the core ideas behind MarS can be adapted from market-order simulation to retail banking customer-event simulation. The prototype shows that a sequence-based financial foundation model can

support short-horizon distress warning, future-path forecasting, intervention what-if analysis, portfolio monitoring, and collections prioritization within a single coherent system.

The project also demonstrates technical understanding across data design, sequence modeling, simulation, downstream applications, governance, and economic framing. Most importantly, the latest results show that the transformer is now the strongest live model in the implemented system, which strengthens the MarS-inspired design choice and gives the prototype a credible empirical story for the MH6818 project.

References

Allen, L., DeLong, G. and Saunders, A., 2004. Issues in the credit risk modeling of retail markets. Journal of Banking & Finance, 28(4), pp.727-752.

Barboza, F., Kimura, H. and Altman, E., 2017. Machine learning models and bankruptcy prediction. Expert systems with applications, 83, pp.405-417.

Barrell, R., Davis, E.P., Karim, D. and Liadze, I., 2010. Bank regulation, property prices and early warning systems for banking crises in OECD countries. Journal of Banking & Finance, 34(9), pp.2255-2264.

Beltman, J. et al. (2025) Predicting retail customers' distress in the finance industry: An early warning system approach. Journal of retailing and consumer services. [Online] 82.

Bequé, A. and Lessmann, S., 2017. Extreme learning machines for credit scoring: An empirical evaluation. Expert Systems with Applications, 86, pp.42-53.

Boonman, T.M., Jacobs, J.P., Kuper, G.H. and Romero, A., 2017. Early warning systems with real-time data.

Kočenda, E. and Vojtek, M., 2009. Default predictors and credit scoring models for retail banking.

Leow, M. and Crook, J., 2014. Intensity models and transition probabilities for credit card loan delinquencies. European Journal of Operational Research, 236(2), pp.685-694.